EDR EVASION & DETECTION

Home Lab — Proof of Concept Report

Author: Vitalii Bibikov

Date: August 2026

Classification: Home Lab / Research Only

1. Introduction

1.1 Lab Environment

This report documents a home cybersecurity lab built to explore Endpoint Detection and Response (EDR) capabilities and evasion techniques. The lab was constructed from freely available, open-source components and runs entirely on a single physical machine The title of the EDR was intentionally omitted to reduce the risk of possible sensitive info leak.

Virtualisation: VirtualBox — Windows VM with Bridged Adapter networking

EDR Stack: EDR Platform — Manager + Indexer + Dashboard deployed via Docker (single-node)

Endpoint Agent: EDR Agent on Windows VM

Telemetry: Sysmon (SwiftOnSecurity config) forwarding to EDR via eventchannel

1.2 Objectives

The lab was set up with three primary objectives:

- Deploy a fully functional, self-hosted EDR stack at zero cost using open-source tooling.
- Analyze EDR's behaviour.
- Systematically develop and iterate a Proof-of-Concept (PoC) that bypasses each detection layer, documenting what evades detection and why.

1.3 Detection Rules Under Test

The following two EDR rules (sourced from 0800-sysmon_id_1.xml) were the primary targets of this research:

Rule 92052 — Windows command prompt started by an abnormal process

Fires on any Sysmon Event ID 1 where cmd.exe is launched by a parent process not in the allowlist (explorer.exe or cmd.exe itself). Uses negate="yes" to flag any parent outside the two permitted entries.

```
<rule id="92052" level="4">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.originalFileName"
         type="pcre2">(?i)cmd\.EXE</field>
  <field name="win.eventdata.parentImage"
```

```
        type="pcre2" negate="yes">(?i)(explorer|cmd)\.EXE</field>
  <options>no_full_log</options>
  <description>Windows command prompt started by an abnormal
process</description>
  <mitre>
    <id>T1059.003</id>
  </mitre>
</rule>
```

Rule 92032 — Suspicious Windows cmd shell execution

Fires on Sysmon Event ID 1 when known reconnaissance binaries (whoami.exe, hostname.exe, etc.) appear as child processes. Matches on the image path and originalFileName fields directly — independent of the parent process.

```
<rule id="92032" level="3">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.image"

type="pcre2">(?i)\\(whoami|hostname|net|ipconfig|systeminfo|tasklist)\.exe</fie
ld>
  <options>no_full_log</options>
  <description>Suspicious Windows cmd shell execution</description>
  <mitre>
    <id>T1087</id>
    <id>T1059.003</id>
  </mitre>
</rule>
```

2. PoC Creation and Bypass

The PoC was developed iteratively — each version targeted a specific detection layer, confirmed the evasion in the EDR dashboard, then moved to the next. All versions were compiled with MinGW/GCC on the Windows VM and compressed with upx to bypass Windows Defender checks:

```
gcc <file>.c -o <file>.exe -masm=intel -Wall -lntdll
upx --ultrabrute <file>.exe
```

2.1 Technique Overview

Version	Key Technique	Rule 92052	Rule 92032	Alerts
v1	HWBP + direct syscall + cmdline obfuscation	Fires	Fires	2 rules
v2	v1 + PPID spoof (explorer.exe)	Silent	Fires	1 rule
v3	v2 + no child process (env vars only)	Silent	Silent	0 alerts

2.2 Detailed Technique Breakdown

Testing began with a PoC using broad evasion techniques to fingerprint the EDR's detection behaviour, then progressively narrowed to target and bypass each specific detection mechanism.

v1 — Initial version: HWBP + Direct Syscall + Command Line Obfuscation

The first full-stack PoC combined three evasion primitives into a single payload. Hardware breakpoints (DR0 via SetThreadContext) replaced the INT3 software breakpoint, eliminating the EXCEPTION_BREAKPOINT event signature. Process creation was performed via NtCreateUserProcess through a direct syscall stub — the syscall number (SSN) was resolved at runtime using Hell's Gate (reading ntdll's export stub bytes), with Halo's Gate as a fallback for hooked stubs. Sysmon's kernel driver still captured the process creation event (confirming kernel-level visibility is not affected by user-mode API bypasses), and both rules fired. The combined techniques established the baseline for further evasion.

v2 — PPID Spoofing (Rule 92052 Eliminated)

Rule 92052's parent allowlist contains only two entries: explorer.exe and cmd.exe. PPID spoofing works by opening a handle to a running explorer.exe instance with PROCESS_CREATE_PROCESS access, then passing that handle as PS_ATTRIBUTE_PARENT_PROCESS in the NtCreateUserProcess attribute list. The kernel records explorer.exe's PID as the new process's parent in the PCB. Sysmon reads the PCB — not the actual calling process — and logs explorer.exe as parentImage. The rule's negate="yes" condition matches the allowlist entry and suppresses the alert. Rule 92052 produced zero alerts for the remainder of testing.

```
static HANDLE FindAndOpenExplorer(void) {
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE) return NULL;

    PROCESSENTRY32W pe = { .dwSize = sizeof(pe) };
    HANDLE hExplorer = NULL;
```

```
    [REDACTED :D]

    CloseHandle(hSnap);
    return hExplorer;
}
```

v3 — No Child Process Spawning (Rule 92032 Eliminated)

Rule 92032 fired on Sysmon Event ID 1 generated by whoami.exe and hostname.exe as child processes of cmd.exe. The fix was to eliminate those child processes entirely. cmd.exe built-in commands and environment variable expansions (%USERNAME%, %COMPUTERNAME%, %USERDOMAIN%, dir) execute inside the existing cmd.exe process with no fork and no new PID. Sysmon generates exactly one Event ID 1 (for cmd.exe itself) and nothing beneath it. With explorer.exe as the spoofed parent and no suspicious child processes, both rules produced zero alerts.

```
static BOOL SpawnNoChildProcess(HANDLE hFakeParent, const wchar_t* cmdline,
                                const char* label) {
    printf("\n[VEH] Spawning: %s\n", label);
    wprintf(L"[VEH] Cmdline: %s\n", cmdline);

    HMODULE ntdll = GetModuleHandleA("ntdll.dll");
    RtlInitUnicodeString_t RtlInitUnicodeString =
        (RtlInitUnicodeString_t)GetProcAddress(ntdll, "RtlInitUnicodeString");
    RtlCreateProcessParametersEx_t RtlCreateProcessParametersEx =
        (RtlCreateProcessParametersEx_t)GetProcAddress(ntdll,
            "RtlCreateProcessParametersEx");

    [REDACTED :D]

    return TRUE;
}
```

2.3 Final PoC Output

The v3 payload wrote the following reconnaissance output to C:\Windows\Temp\v3out.txt with zero EDR alerts generated:

```
user=vboxuser  computer=EDR  domain=EDR  arch=AMD64

Volume in drive C has no label.
Volume Serial Number is AE8C-0160

Directory of C:\Users
08/08/2026  03:46 PM    <DIR>    Public
08/08/2026  05:10 PM    <DIR>    vboxuser
               0 File(s)       0 bytes
               4 Dir(s)   20,847,898,624 bytes free
```

3. How to Defend Against It

Each evasion technique exploited a specific gap in the detection configuration. The following recommendations address those gaps directly, ranging from immediate EDR rule improvements to deeper architectural changes.

3.1 Detection Gaps and Fixes

Detection Gap	Recommended Fix
PPID spoofing evades parent-child rules	Correlate grandparent PID: flag when explorer.exe's own parent is a non-system binary
No child process — env var recon invisible	Monitor cmd.exe instances with no child but writing to disk (Sysmon Event ID 11)
Direct syscall bypasses user-mode hooks	Kernel ETW (Event Tracing for Windows) telemetry captures syscalls regardless of API path
RWX memory page for syscall stub	Alert on VirtualAlloc with PAGE_EXECUTE_READWRITE from non-system processes
Hardware breakpoints (DR registers)	Monitor SetThreadContext calls that modify DR0-DR3/DR7 from user-mode processes
Caret obfuscation breaks content rules	Normalise command lines before matching: strip carets, expand %vars%, lower-case

3.2 EDR Rule Improvements

Expand the rule 92052 parent allowlist

The current allowlist (explorer.exe and cmd.exe) is too narrow and too easily spoofed. A more robust approach adds grandparent correlation — checking not just parentImage but whether the reported parent was itself spawned by a known-good process. Add a chained rule that fires when explorer.exe spawns cmd.exe but the explorer.exe instance was created by an unusual process

Detect RWX memory allocation

The direct syscall stub requires a PAGE_EXECUTE_READWRITE memory region. Sysmon Event ID 8 (CreateRemoteThread) does not capture this, but a custom Sysmon configuration can monitor for VirtualAlloc calls with suspicious protection flags. Alternatively, add an EDR rule that fires when a non-system process allocates RWX memory, using Sysmon's process access events or a memory scanning integration.

3.3 Architectural Recommendations

Deploy kernel ETW telemetry

Event Tracing for Windows (ETW) captures syscall activity at the kernel level, independent of which API path was used. Direct syscalls that bypass ntdll hooks are still visible via ETW. Integrating an ETW consumer (such as SilkETW or a commercial solution) alongside the EDR/Sysmon stack closes the user-mode blind spot exposed in v1.

Monitor file writes from cmd.exe instances

The v3 payload wrote reconnaissance output to C:\Windows\Temp\v3out.txt. Sysmon Event ID 11 (File Create) would log this. An EDR rule alerting on file creation in Temp by cmd.exe instances with no prior child process events in the same session would catch v3's file-based output even when process creation events are clean.

Consider an EDR with a call stack analysis

EDR products such as CrowdStrike Falcon, SentinelOne, analyse the call stack at the moment of a syscall. A SYSCALL instruction with no ntdll frame above it is a strong anomaly signal that indicates a direct syscall bypass. This capability is not available in Sysmon alone and represents the primary advantage of those solutions for detecting direct syscall evasion.

4. Summary

This lab demonstrated a complete, iterative approach to EDR research: building a detection stack, validating it against real techniques, then systematically evading each detection layer to understand its exact scope and limitations.

The EDR + Sysmon stack proved effective against naive process execution and reconnaissance. Rules 92052 and 92032 provided solid behavioural and content-based coverage for standard techniques. However, combining PPID spoofing with the elimination of child process spawning was sufficient to reduce the alert count to zero — highlighting that detection rules based solely on parent-child relationships and binary execution patterns have a well-defined ceiling.

The key insight from this research is that Sysmon's kernel-level telemetry is robust against user-mode API evasion (direct syscalls, ntdll unhooking) but relies on the accuracy of the data the kernel reports — which can be influenced by PPID spoofing at the process creation level. Closing this gap requires either rule-level correlation of process genealogy beyond the immediate parent, or kernel ETW telemetry that is independent of the reported parent PID.

4.1 Key Findings

- Sysmon (kernel driver) captures process creation events regardless of whether CreateProcess or NtCreateUserProcess is used — direct syscalls do not bypass kernel-level telemetry.
- Rule 92052's parent allowlist (explorer.exe and cmd.exe only) is insufficient against PPID spoofing. Grandparent correlation or session-level process tree analysis is required.
- Eliminating child process spawning (using cmd.exe built-ins and environment variables) removes the primary signal that rule 92032 relies on.
- Command line obfuscation (carets, delayed expansion) bypasses content-matching rules that do not normalise input before applying regex patterns.
- The combination of PPID spoofing + no child processes + HWBP + direct syscall produced zero EDR alerts while successfully exfiltrating host reconnaissance data.